

Group C: Reinforcement-Learning Dispatch of a 1 MWh Battery

Udval Oyunsai Khan, Andres Cadena, Sebastian Panesso

2025-07-22

1. Introduction

This project aims to develop and evaluate an intelligent agent for the optimal dispatch of a 1 MWh battery storage system operating in the Austrian electricity market. We leverage a Deep Q-Network (DQN), a prominent Reinforcement Learning (RL) algorithm, to learn a profitable trading strategy from historical price data. The performance of the trained RL agent is benchmarked against two simpler, heuristic-based policies to quantify the value of its learned strategy.

2. Methodology

2.1. Data and Preprocessing

The analysis is based on historical 15-minute day-ahead electricity spot prices for the Austrian market, spanning from January 1, 2019, to December 31, 2024. The raw data consists of timestamped price entries in UTC. Our preprocessing pipeline involved the following steps:

- **Time Zone Conversion:** The native UTC timestamps were converted to the local Central European Time (CET) zone to align with the operational context of the Austrian market.
- **Chronological Sorting:** The dataset was sorted chronologically to ensure the correct temporal sequence for time-series analysis.
- **Integrity Check:** The dataset was checked for missing values. No missing observations were found, so no imputation was necessary.
- **Train-Test Split:** To ensure a robust evaluation that prevents data leakage, the dataset was split chronologically. Data from **2019 through 2022** was allocated for training the Reinforcement Learning agent, while data from **2023 through 2024** was held out as an unseen test set for evaluating all models.

2.2. Battery Simulation Environment

A discrete-time simulation environment was developed to model the physical and economic characteristics of a 1 MWh containerized lithium-ion battery. At each 15-minute time step, an agent can select one of three actions: charge, discharge, or hold. The environment enforces the following constraints as specified in the task:

- **Energy Capacity:** The battery's State of Charge (SoC), represented as a value from 0.0 to 1.0, is constrained implicitly. The `min()` function is used to ensure that the energy charged or discharged never violates the 0% or 100% boundaries. For example, during discharge, the amount of energy moved is capped by `min(env$soc, energy)`, preventing the SoC from becoming negative.

- **Power Limit:** The maximum charge or discharge rate of 0.5 MW is enforced by capping the energy transfer per 15-minute step at `POWER_LIMIT_MW15` (0.125 MWh).
- **Round-Trip Efficiency:** The model implements a 90% one-way efficiency factor, which differs from a 90% round-trip efficiency.
 - During Charging: The energy added to the SoC is 90% of the maximum possible based on the power limit ($\text{energy} * \text{EFFICIENCY}$). The cost, however, is calculated based on the gross energy drawn from the grid ($\text{charge_possible} / \text{EFFICIENCY}$), accurately reflecting that the operator pays for energy that is lost as heat.
 - During Discharging: The energy sold to the grid is 90% of the energy taken from the battery ($\text{discharge_possible} * \text{EFFICIENCY}$), modeling the conversion loss.
- **Degradation Cost:** A cost of €20 per MWh of throughput (both for charging and discharging) is applied to model battery wear and tear.

The environment is implemented as an isolated object using `new.env(parent = emptyenv())`. The `parent = emptyenv()` argument ensures the environment does not inherit variables from R’s global environment, preventing unintended interactions and keeping the simulation self-contained and reproducible. The agent interacts with the environment through the `step_env()` function. All operational limits are enforced within this function, using the `min()` function to guarantee that the battery does not violate physical constraints. This provides a realistic framework for the reinforcement learning agent to learn profitable strategies while strictly adhering to the battery’s physical and economic characteristics.

2.3. Benchmark Models

To establish a performance baseline, two heuristic-based models were implemented. These models operate under the same environmental constraints as the RL agent.

- **Random Policy:** This policy serves as a naive baseline. At each time step, it selects an action (hold, charge, or discharge) with equal probability (1/3), without regard for the current price or the battery’s state of charge.
- **Rule-Based Policy:** This policy implements a simple, intuitive trading heuristic. Price thresholds govern the decision logic. To ensure the model is robust and avoids data leakage, these thresholds were determined non-parametrically based on the price distribution of the training data (2019-2022). The charge threshold was set to the 20th percentile and the discharge threshold was set to the 80th percentile of this historical data. This resulted in the following operational rules:
 - Charge: If the current market price is below 30.96.
 - Discharge: If the current market price is above 194.92.
 - Hold: If the price is between these two thresholds.

2.4. Reinforcement Learning Agent (DQN)

To address the sequential decision-making problem of battery dispatch, we implemented a Deep Q-Network (DQN) agent. The agent’s objective is to learn a policy that maximizes the cumulative discounted reward by learning an optimal action-value function, $Q^*(s, a)$. The agent was trained for 50 episodes on a dataset of 15,000 observations.

- **Q-Network Architecture:** The core of our agent is a neural network that approximates the Q-function. It takes a state vector as input and outputs a Q-value for each possible action (Hold, Charge, Discharge). Our architecture consists of an input layer matching the state dimension (97 units: 96 for the historical price window and 1 for the SoC), two hidden layers with 128 units each using the ReLU activation function, and a linear output layer with 3 units.

- **State and Action Space:** The state (s) includes the battery’s current SoC and a look-back window of the previous 24 hours of normalized electricity prices. The action (a) space consists of three discrete actions: Hold (0), Charge (1), or Discharge (2).
- **Experience Replay & Target Network:** To stabilize training, we utilize an experience replay buffer with a capacity to store 10,000 past experiences and a separate target network. The agent begins its learning process after an initial “warmup” period where it populates the buffer with 1,000 experiences. At each step, the experience tuple (s, a, r, s') is stored. Mini-batches of size 128 are sampled randomly from this buffer to update the network. The target network, a copy of the main Q-network updated every 10 episodes, provides a stable learning target based on the Bellman equation using a discount factor γ of 0.99:

$$y_i = r_i + \gamma \max_{a'} Q_{target}(s'_i, a').$$

- **ϵ -Greedy Exploration:** To balance exploration and exploitation, the agent employs an ϵ -greedy policy. The value of ϵ decays exponentially from 1.0 to 0.05 over the training episodes.

2.5. T4 Enhancement: Double DQN

A well-known issue with the standard DQN algorithm is its tendency to overestimate action values. This “maximization bias” occurs because the $\max$ operator uses the same network to both select the best action and evaluate its value. To mitigate this, we implemented the Double Deep Q-Network (DDQN).

The core innovation of DDQN is the decoupling of action selection from action evaluation. The target Q-value calculation is modified to use both the main and target networks:

$$y_i = r_i + \gamma Q_{target}(s'_i, \arg \max_{a'} Q_{main}(s'_i, a'))$$

First, the main Q-network selects the best action for the next state. Then, the target network evaluates the Q-value of that specific action. This reduces the risk of consistently picking an action whose value is overestimated, leading to more stable Q-value estimates and often resulting in better overall policy performance.

3. Results

This section presents the final, quantitative outcomes of all models when evaluated on the unseen test set (2023-2024).

3.1. Benchmark Performance

The two benchmark policies were simulated over the entire test period. Their final cumulative profits are presented in Table 1.

Table 1: Final Cumulative Profit of Benchmark Models on Test Data.

Policy	Final.Profit...
Random Policy	-152546.71
Rule-Based Policy	7446.07

3.2. RL Agent Performance

The trained Reinforcement Learning agents were evaluated on the same test set. Table 2 summarizes the performance of all models, providing a direct comparison.

Table 2: Overall Performance Comparison on Test Data (2023-2024).

Model	Final.Profit. . .
Random Policy	-152546.71
Rule-Based Policy	7446.07
Baseline DQN Agent (T1)	18644.12
Enhanced DDQN Agent (T4)	9875.01

4. Evaluation and Analysis

This section provides an in-depth interpretation of the results, focusing on the comparative behavior of the agents.

4.1. Comparative Strategy Analysis

A visual comparison of the agents’ strategies over the first week of the test period reveals the source of the DQN agent’s superior performance.

The Rule-Based Agent (Figure 1) exhibits a predictable and rigid strategy. It consistently charges the battery to full capacity whenever the price drops below its fixed 194.92 threshold (e.g., during the night of Jan 01-02) and fully discharges as soon as the price exceeds its 30.96 threshold. This strategy is profitable but lacks nuance; it treats every low price and every high price as an equal opportunity, failing to prioritize the most profitable spreads.

In stark contrast, the Baseline DQN Agent (T1) (Figure 2) demonstrates a much more sophisticated, opportunistic, and context-aware policy. Its key advantages are:

- **Selective Trading:** Unlike the Rule-Based agent, the DQN agent does not always charge to 100% or discharge to 0%. On Jan 03, it correctly identifies a massive price spike and discharges its full capacity. However, during the smaller price peaks on Jan 05 and Jan 06, it only performs partial discharges or holds its charge entirely. This indicates the agent has learned to estimate the potential of a price swing and saves its energy for truly high-profit opportunities, avoiding the degradation costs associated with marginal trades.
- **Anticipatory Inaction:** From Jan 05 to Jan 08, the price volatility decreases. The DQN agent recognizes this and dramatically reduces its trading activity, keeping its State of Charge (SoC) very low. It correctly learns that in a low-spread environment, the cost of degradation outweighs the potential arbitrage profit, so the best action is to do nothing. The Rule-Based agent, bound by its fixed rules, continues to perform small, inefficient charge-discharge cycles during this period.

To understand the strategic differences, we visualize the behavior of the Rule-Based agent and the two RL agents over the same one-week period.

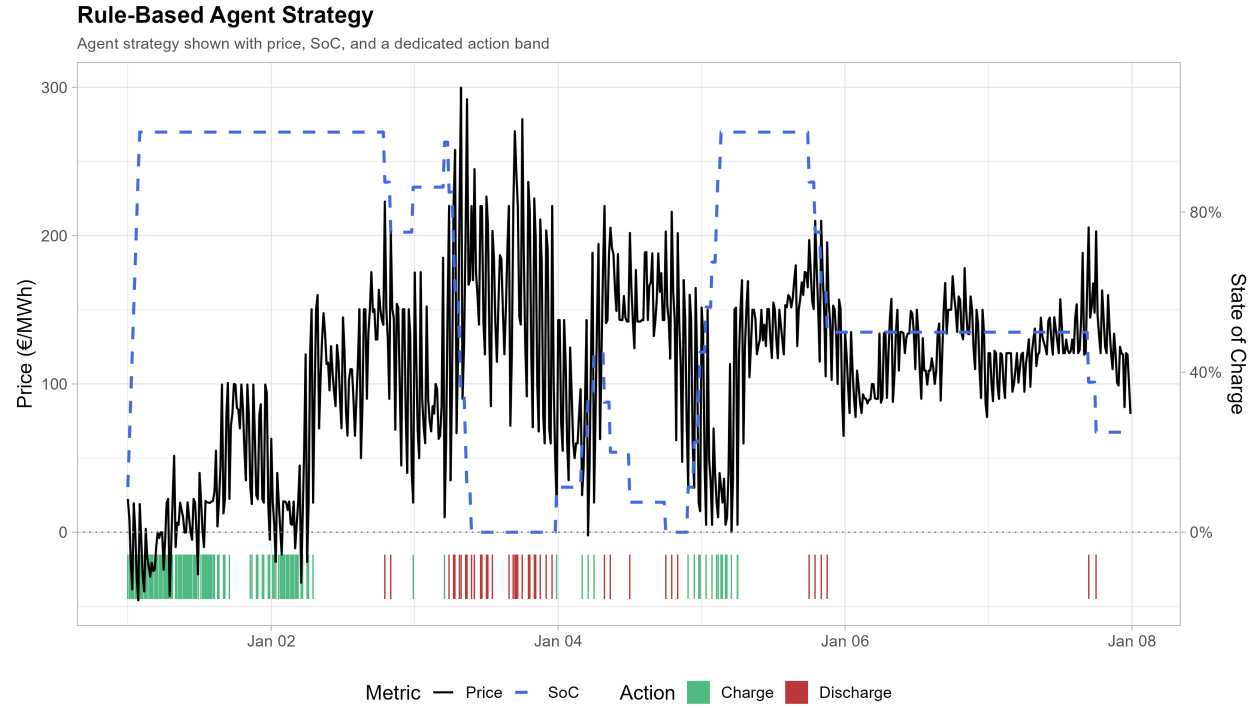


Figure 1: Behavior of the Rule-Based Agent.



Figure 2: Behavior of the DQN Agent.

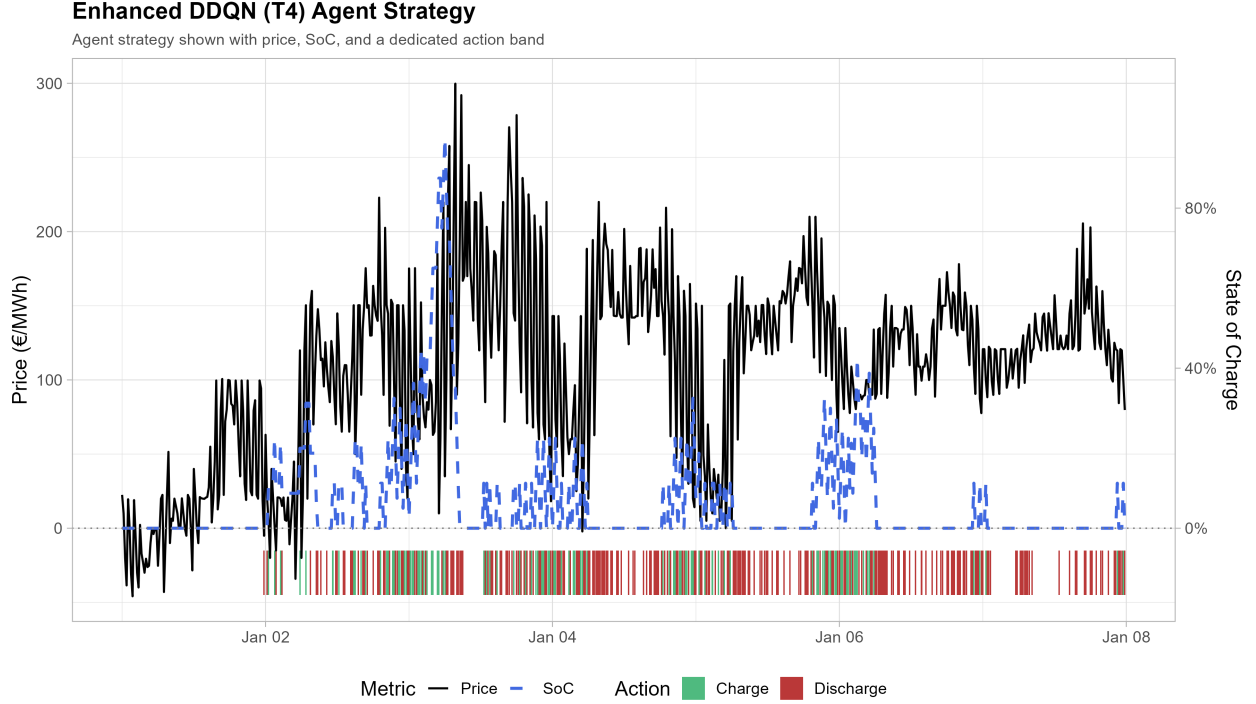


Figure 3: Behavior of the DDQN Agent.

4.2. Impact of T4 Enhancement

To improve our model, a Double DQN (DDQN) agent was trained under the same conditions as the baseline DQN model. While the DDQN agent’s profit of €9,875.01 was a significant improvement over the Rule-Based policy, it was notably lower than the baseline DQN’s profit of €18,644.12. A direct visual comparison of their strategies (Figure 2 vs. Figure 3) and action frequencies (Figure 4) reveals the reason for this discrepancy.

Aggression vs. Caution: The baseline DQN (Figure 2) adopts a hyper-aggressive strategy, visible in its extremely dense action band. It frequently alternates between small charges and discharges, attempting to capture even minor price fluctuations. The DDQN agent (Figure 3), by contrast, is visibly more cautious. Its action band is sparser, indicating longer periods of holding and fewer overall trades. This is quantified in Figure 4, which shows the DDQN performed fewer charge and discharge actions. **Different Responses to Volatility:** During the highly volatile period from Jan 02 to Jan 04, the baseline DQN’s aggressiveness paid off, allowing it to capitalize on rapid price swings. However, the DDQN’s more conservative nature led it to hold its charge more often, missing some of these short-term opportunities.

This finding suggests that for this specific problem and training duration, the standard DQN’s known tendency to overestimate action values may have led it to adopt a risk-taking strategy that was particularly effective on the volatile test data. The DDQN’s more stable learning process, while theoretically superior, resulted in a less aggressive policy that was ultimately less profitable in this market environment.

Furthermore, the DDQN’s training log showed a consistent upward trend in profit even at the final episode, indicating the agent had not yet fully converged. It is plausible that with a longer training period, the DDQN’s more accurate value estimations would have allowed it to develop a more refined policy and ultimately outperform the baseline model.

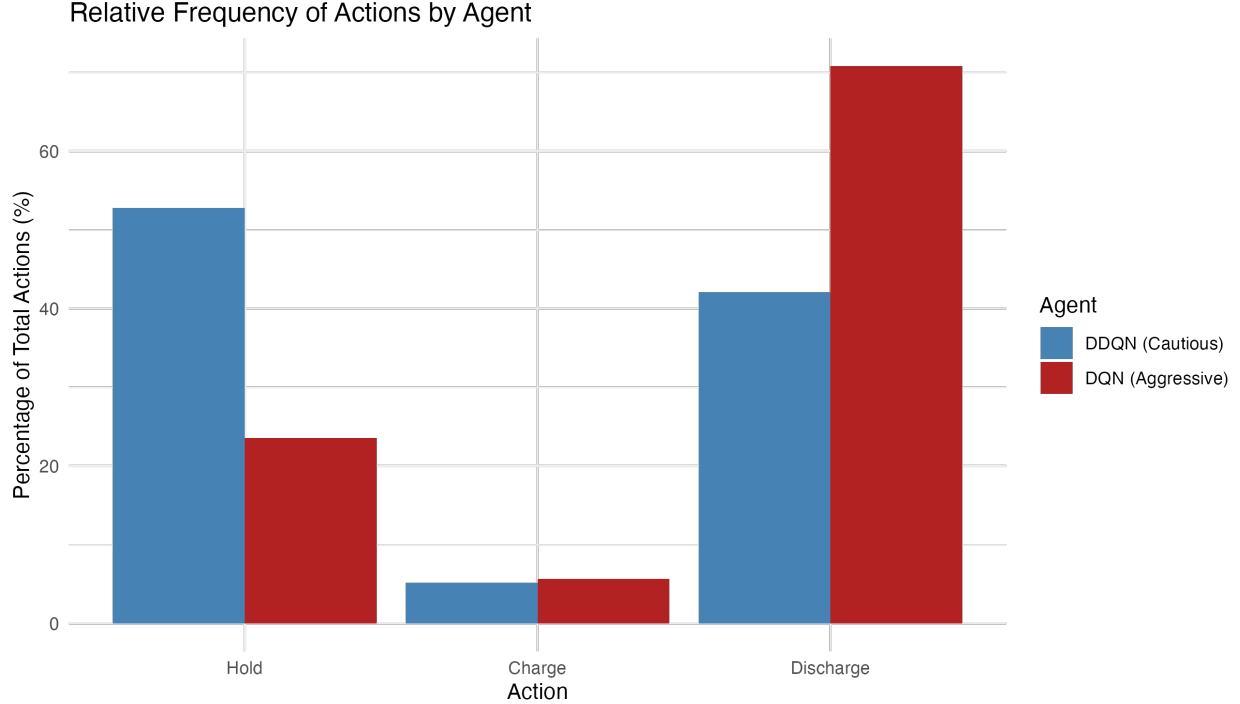


Figure 4: DQN Vs. DDQN

5. Conclusion

This project successfully demonstrated the significant potential of Deep Reinforcement Learning for optimizing the economic dispatch of a 1 MWh battery in the volatile Austrian electricity market. Our primary finding is that a well-trained RL agent can substantially outperform simpler, heuristic-based strategies.

The baseline Deep Q-Network (DQN) agent emerged as the most profitable model, achieving a final profit of €18,644.12 on the unseen test data. This result not only represents a viable business case but also more than doubles the €7,446.07 profit generated by a data-driven Rule-Based policy and stands in stark contrast to the significant losses incurred by a Random policy. The success of the DQN agent stems from its ability to learn a dynamic and context-aware strategy, allowing it to prioritize high-margin trades and prudently avoid action during periods of low price volatility to minimize degradation costs—a level of sophistication that fixed-rule systems cannot achieve.

The investigation into the Double DQN (DDQN) enhancement provided a valuable insight into the nuances of RL training. While the DDQN learned a profitable strategy, its more conservative nature yielded a lower profit than the baseline DQN within the given training constraints. This highlights that theoretical algorithmic improvements do not always guarantee superior performance without sufficient training time or hyperparameter tuning.

In conclusion, our work affirms that Reinforcement Learning is a powerful and highly effective tool for the complex, sequential decision-making task of energy arbitrage. Future work could explore the impact of longer training periods for the DDQN, the inclusion of price forecasts directly into the agent’s state, or the application of more advanced RL algorithms like Proximal Policy Optimization (PPO) to further refine the dispatch strategy.